

アプリ生成ワークフローガイド

仕様化前ノートによるアプリ作成

(株) SRA

Version 0.1.0

2026-05-07

利用者ガイド

このガイドは、`app-generation-workflow` を使ってアプリを作成する人向けの説明書である。

このワークフローでは、アプリをいきなり実装しない。まず任意形式の要求 `.txt` から仕様化前ノートに材料を集め、そこから三点セットを作り、最後に実装方式を指定してアプリを作成する。未決事項を減らしたい場合は、推奨テンプレート

`templates/requirement.txt` を使って要求を整理してから始める。

このガイドは利用者向けの入口である。作業手順の正本は

`three_key_documents_workflow.md` とし、詳細な判断に迷った場合はそちらを参照する。

ワークフローは、ユーザーの作業場所にコピー済みのものだけを参照する。他の場所にあるワークフローは参照しない。コピー済みワークフローはテンプレートとして扱い、対象アプリ固有の内容を書き込まない。

まず読むもの

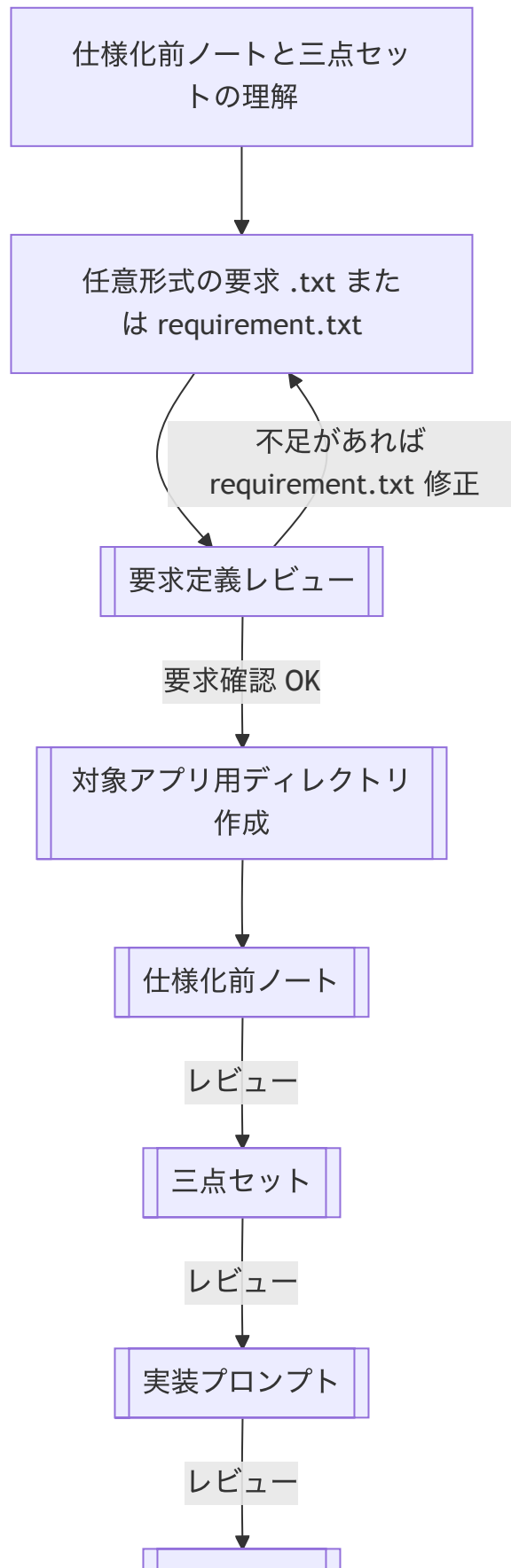
最初にこのガイドを読む。次に、作業手順の正本として次の文書を読む。

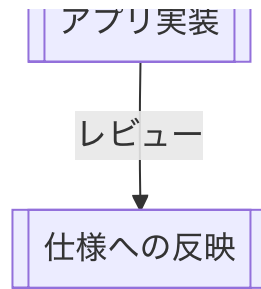
- `three_key_documents_workflow.md`

AI に作業前提を理解させる場合も、次のように指示する。

`app-generation-workflow/three_key_documents_workflow.md` を読んでください。

基本の流れ





1. 仕様化前ノートを作る

ユーザー要求は、任意形式の `.txt` ファイルとして受け取る。要求 `.txt` は生の入力であり、実装用の仕様書ではない。

未決事項を減らしたい場合は、次の推奨テンプレートをコピーして

`requirement.txt` として使う。任意形式の要求でもよいが、

`requirement.txt` を使うと、仕様化前ノートを作る前に不足を見つけやすい。

```
app-generation-workflow/templates/requirement.txt
```

要求 `.txt` または `requirement.txt` を受け取ったら、AI が要求定義レビューを行う。仕様化前ノートを作れるだけの材料があるか、目的、利用者、アプリ種別、入力、表示、操作、状態ごとの操作、操作衝突、OS差分・入力デバイス差分、ルール、対象外、例外、代表例、未決事項が読み取れるかを確認する。

要求定義レビューで不足が見つかった場合、AI はユーザーへ確認した方がよい点、

`requirement.txt` への追記候補、推測で補いそうな箇所を提示する。AI の追記候補は推薦案であり、ユーザーが採用するまで正式な要求として扱わない。不足が実装や三点セット生成に影響する場合は、仕様化前ノートや三点セットへ進まず、

`requirement.txt` を修正する。ここで止まることは、曖昧な要求からそれらしい仕様やコードを作らないための正しい挙動である。

要求定義レビューを通ったら、対象アプリ用の新しいディレクトリを作成する。以後、仕様化前ノート、三点セット、実装プロンプト、ソースコードは、すべて対象アプリ用ディレクトリ配下に置く。

対象アプリの作業ディレクトリへ、次のテンプレートをコピーする。

```
app-generation-workflow/templates/source_memo.md
```

コピー先の例:

```
my-app/source_memo.md
```

コピーした `source_memo.md` に、次の項目を反映する。

- 作りたいアプリの目的
- 利用者
- アプリ種別
- やりたいこと
- 入力する情報
- 表示したい情報
- 守るべきルール
- 状態、処理の流れ
- 状態ごとの操作、操作衝突、操作対象の分類
- 往復操作、OS差分・入力デバイス差分
- 代表例、期待結果
- 作らない機能
- まだ決めていないこと

アプリ種別は、最初に最も近いものを1つ選ぶ。複数の性質がある場合は主目的を1つ選び、補足に残す。最初から正確に分類できない場合は仮選択でよい。

主な種別:

- 入力・計算・結果表示アプリ
- データ管理アプリ
- ビューアアプリ
- エディタアプリ
- 変換ツール
- ゲーム / インタラクティブアプリ
- その他

テンプレート本体には、対象アプリ固有の内容を書き込まない。コピー済みワークフロー本体も変更しない。

未決事項を書く場合は、決定後にどこへ移すか分かるように、各項目へ反映先を1行で付ける。

- 未決事項: 保存方式を JSON、SQLite、CSV のどれにするかを定める。決定後の反映先: 状態・処理の流れ、入力ルール、必要に応じて実装方式

最初に作成した仕様化前ノートに未決事項がある場合は、AI の確認内容に従って判断する。ユーザーが未決事項を「なしにする」と判断した場合は、未決事項欄へ **- なし** と記載する。ユーザーが「なしにしない」と判断した場合は、AI が推薦案を提示する。推薦案が採用または修正された場合は、未決事項欄から削除し、該当する章へ決定内容に移す。

仕様化前ノートを作成したら、要求 `.text` の内容が漏れなく反映されているか、未決事項と対象外が整理されているかを確認する。AI が作成した仕様化前ノートは草案として扱い、人が確認してから三点セットへ進む。

1.1 逆生成レビュー用の仕様化前ノート

既存実装、生成済み三点セット、またはコードから仕様化前ノートを再構成する場合は、通常の `templates/source_memo.md` ではなく、次のテンプレートを使う。

```
app-generation-workflow/templates/source_memo_reverse_review.md
```

コピー先では、通常の仕様化前ノートと同じように `source_memo.md` として配置する。

```
my-app/source_memo.md
```

逆生成では、実装から読み取れる挙動が正しい仕様とは限らない。そのため、現行実装から読み取れる挙動、仕様として採用する挙動、バグ候補・要確認事項を分けて整理する。

主に確認すること:

- 実装由来の挙動を、そのまま正式仕様として書いていないか。
- バグ候補や偶然の挙動を、仕様として採用していないか。
- 境界値や代表例の期待結果に、人の採用判断があるか。
- 未決事項が本文で断定仕様として書かれていないか。

2. 三点セットを作る

仕様化前ノートをもとに、次の三点セットを作成する。

```
my-app/specs/01_usecase_spec.md
my-app/specs/02_ui_spec.md
my-app/specs/03_business_spec.md
```

使用するテンプレート:

```
app-generation-workflow/templates/01_usecase_spec.md
app-generation-workflow/templates/02_ui_spec.md
app-generation-workflow/templates/03_business_spec.md
```

三点セットの役割:

- UC仕様書: 利用者の目的、利用シナリオ、成功条件、例外条件を整理する。
- UI仕様書: 画面、入力項目、表示項目、操作、エラー表示を整理する。
- ビジネスレイヤー仕様書: UIに依存しないデータ、入力検証、計算、保存、更新、削除、状態変更を整理する。

三点セットでは、`source_memo.md` で選んだアプリ種別を引き継ぐ。アプリ種別は固定仕様ではなく、書くべき観点を選ぶための目印である。たとえば、ゲーム / インタラクティブアプリでは状態遷移や終了条件を重視し、変換ツールでは入力解析、変換、出力、失敗条件を重視する。

テンプレート内の任意パターンは、必要な場合だけ使う。管理画面、追加・削除、状態管理、集計、保存・読み込みが不要なアプリでは「対象外」または「なし」と記載する。

2.1 三点セットをレビューする

実装に進む前に、三点セットをレビューする。AI が生成した三点セットは、生成直後は草案として扱う。人がレビューし、必要な修正を反映した後にだけ、レビュー済み三点セットとして扱う。レビュー済みでない三点セットから、実装プロンプトやソースコードを作成しない。

レビューは、アプリの規模とリスクに応じて軽量レビュー、標準レビュー、重点レビューを使い分ける。

軽量レビュー:

- 小さいアプリ、試作、影響範囲が狭い変更で使う。
- 空欄、未決事項、明らかな矛盾を確認する。

標準レビュー:

- 通常の新規アプリ、機能追加、仕様変更で使う。
- 三点セット間の対応、仕様化前ノートとの対応、対象外、テスト観点を確認する。

重点レビュー:

- 既存コードからの逆生成、別言語移植、業務ルールが重いアプリ、失敗時の影響が大きい機能で使う。
- 保存、削除、課金、個人情報、権限、外部連携、安全性、移植リスクを確認する。
- バグ候補や偶然の挙動を正式仕様として採用していないかを確認する。

レビューでは、AI による補助確認を使う。AI には、形式、空欄、矛盾、抜け、推測、未決事項、バグ候補の列挙を行わせる。人は、AI が列挙した確認事項をもとに、採用判断、業務判断、安全性、対象外、移植方針を決める。

3. 仕様化前ノートを分割する場合

通常は、単一の `source_memo.md` から始める。ただし、次のどちらかに当てはまる場合は、5分割仕様化前ノートを使う。

1. 最初から規模が大きいと分かっている場合。
2. 単一 `source_memo.md` で始めたが、途中で大きくなった場合。

3.1 最初から規模が大きい場合

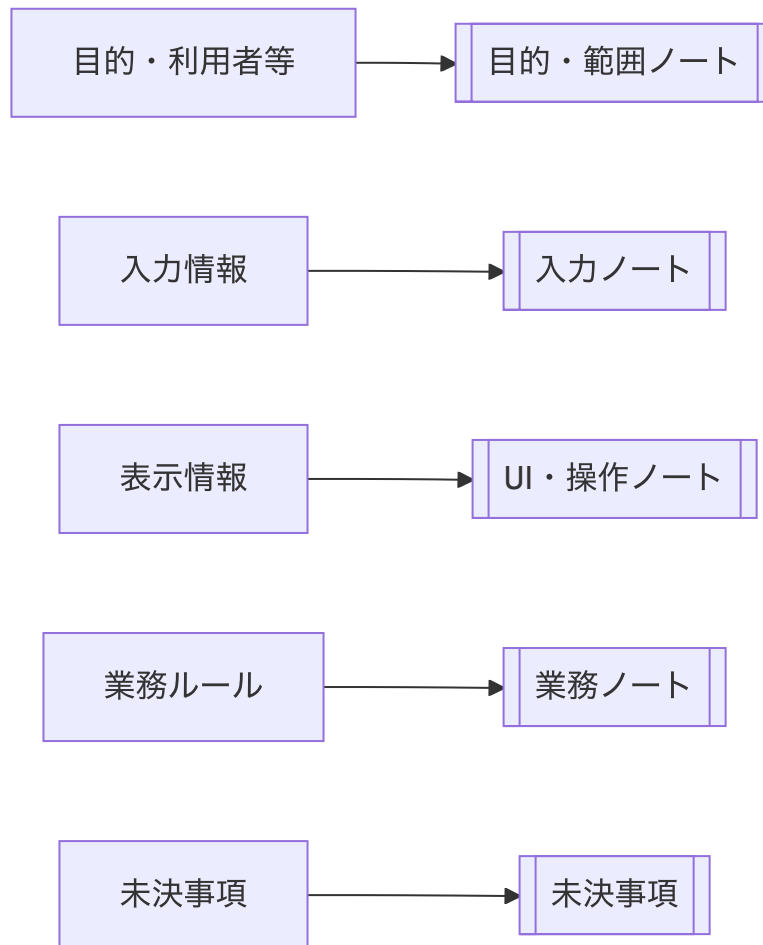
最初から画面数、ユースケース、業務ルール、入力項目、例外条件が多いと分かっている場合は、単一 `source_memo.md` を作らず、最初から `five_source_memos/` を作る。この場合、ワークフロー上の正本は `source_memo.md` ではなく、`five_source_memos/` 配下の分割ノート群とする。

構成例:

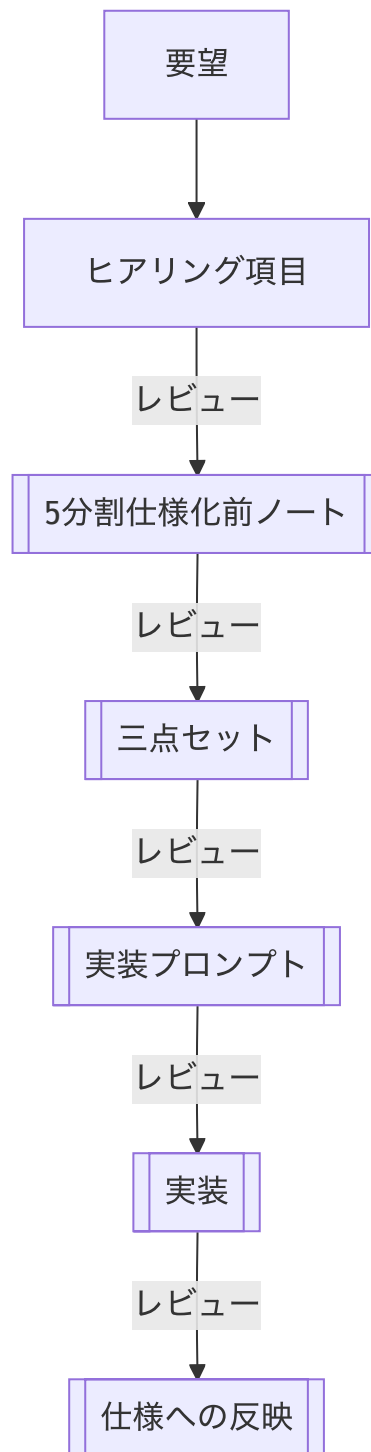
```
five_source_memos/  
├─ index.md  
├─ shared.md  
├─ 01_purpose_scope.md  
├─ 02_input.md  
├─ 03_business_logic.md  
├─ 04_ui_operation.md  
└─ 05_test_examples.md
```

ヒアリング9項目の反映先:

図中の「目的等」は、目的、利用者、アプリ種別、やりたいこと、作らない機能を表す。



この場合の流れ:

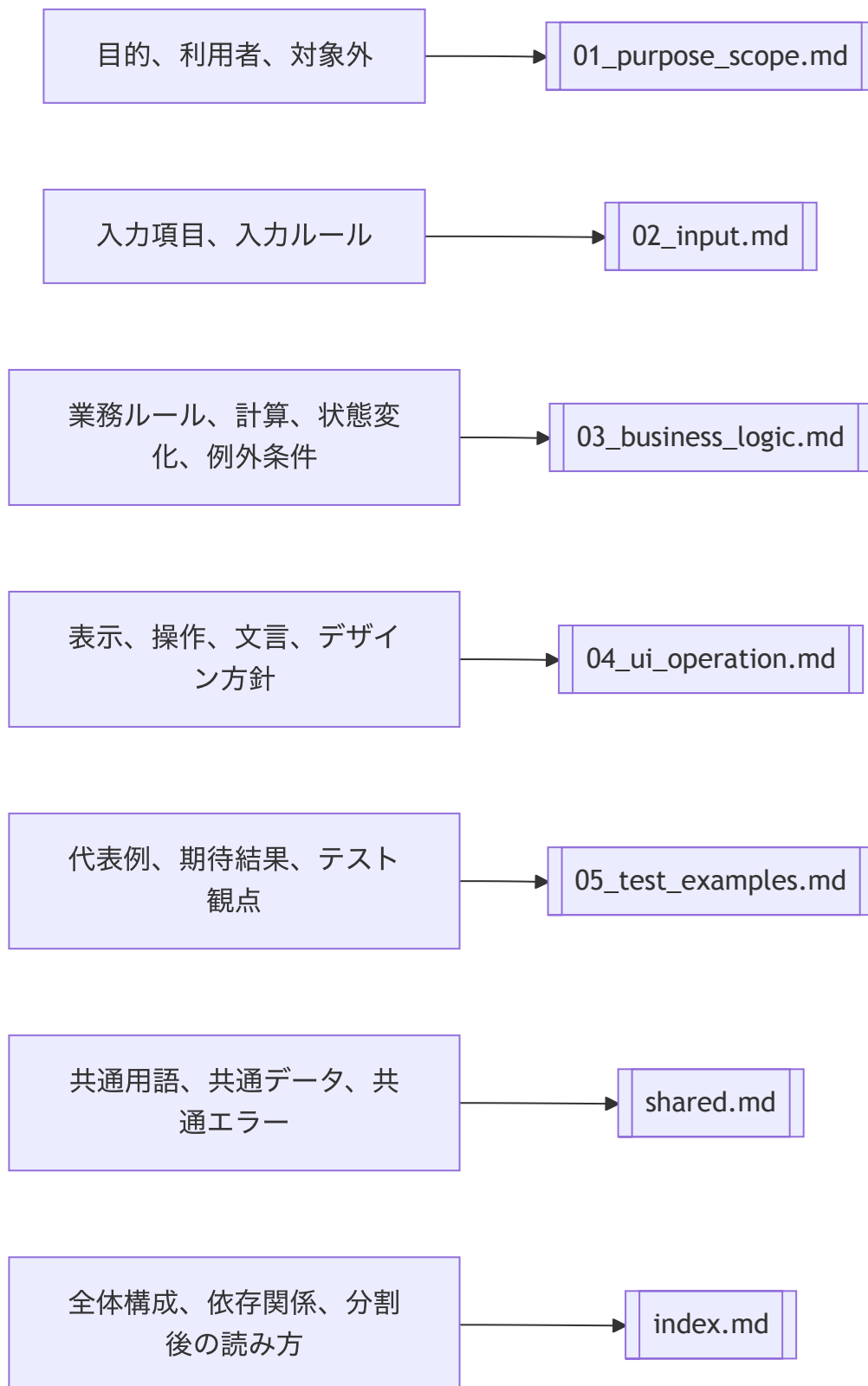


3.2 途中で大きくなった場合

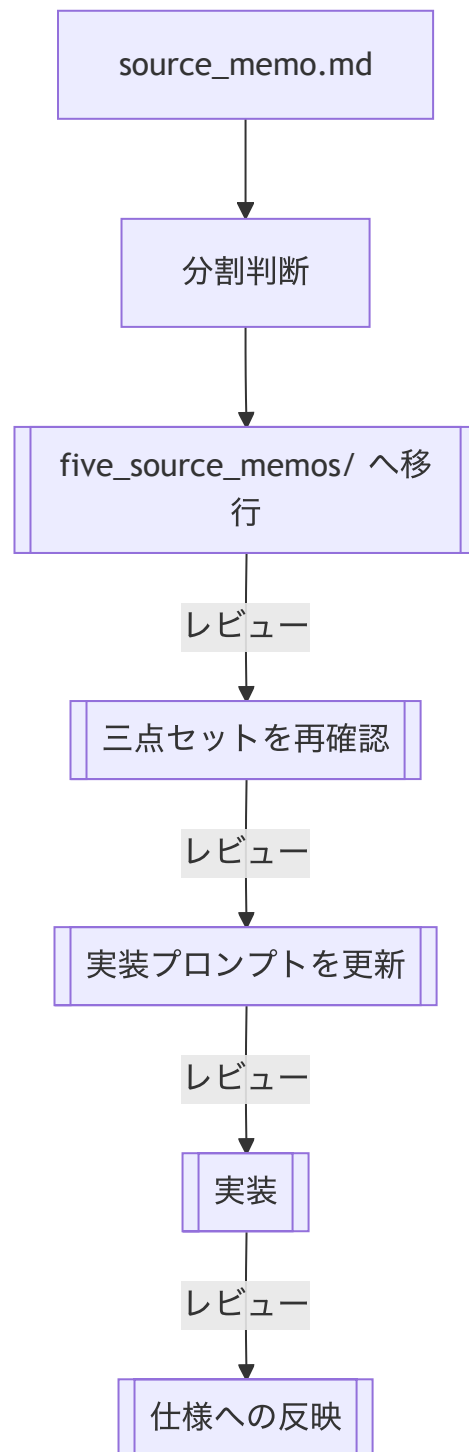
最初は単一 `source_memo.md` で始めてよい。ただし、内容が増えて見通しが悪くなった場合は、`five_source_memos/` に移行する。

途中で分割する場合は、既存の `source_memo.md` を直接捨てない。まず `five_source_memos/` を作り、内容を役割ごとに移す。

移行先:



この場合の流れ:



移行後は、単一 `source_memo.md` を正本として扱わない。正本は `five_source_memos/` 配下の分割ノート群とする。以後の修正、三点セット作成、実装プロンプト更新では、`source_memo.md` ではなく `five_source_memos/` を参照する。

分割した場合も、意図違いの修正時に三点セットやコードを直接修正しない。まず該当する仕様化前ノートを修正し、その後に三点セット、実装プロンプト、コードへ反映する。

5分割仕様化前ノートへ移行したら、移行漏れ、重複、意味の変化、未決事項の消失、共通用語の整理、正本の切り替えを確認する。

4. 実装プロンプトを作る

レビュー済み三点セットを確認した後、実装方式を決める。AI が生成したままの三点セット、未決事項が残る三点セット、採用判断が終わっていない逆生成三点セットからは、実装プロンプトを作成しない。

例:

- Qt デスクトップアプリ
- Web アプリ
- CLI ツール
- モバイルアプリ
- API サーバー

次のテンプレートをコピーして、対象アプリ用の実装プロンプトを作成する。

```
app-generation-workflow/templates/implementation_prompt.md
```

分割した仕様化ワークフロー全体に共通するビルド互換性、コード生成方針、移植性などの実装制約がある場合は、次のテンプレートを対象アプリ用ディレクトリへコピーして使う。この内容は三点セットへ混ぜず、実装プロンプト作成時に参照する。仕様化前ノートの入力ルールには、業務上の制約や入力制約を記載し、ビルド互換性やコード生成方針のような実装制約は混ぜない。

```
app-generation-workflow/templates/implementation_constraints.md
```

コピー先の例:

```
my-app/prompts/implementation_prompt.md
my-app/implementation_constraints.md
```

実装プロンプトには、三点セットを上位仕様として扱うことを明記する。ここで参照する三点セットは、レビュー済みである必要がある。実装時の作業ディレクトリは対象アプリ用ディレクトリとし、`source_memo.md`、`specs/`、`implementation_constraints.md` をその作業ディレクトリから参照する。`implementation_constraints.md` が存在する場合は、プロジェクト共通の実装制約として実装プロンプトへ参照を記載する。

実装プロンプトを作成したら、レビュー済み三点セットを参照しているか、実装方式、ビルド方式、テスト方式、対象外、未決事項が揃っているかを確認する。

`implementation_prompt.md` が使える状態になっていない場合は、実装へ進まない。未作成、空欄あり、参照先の仕様不足、実装方式未記入、実装前に確認すべき未決事項あり、のいずれかに当てはまる場合は、不足している内容を警告として列挙する。

5. アプリを実装する

実装では、三点セットと実装プロンプトを上位仕様として扱う。

実装時の原則:

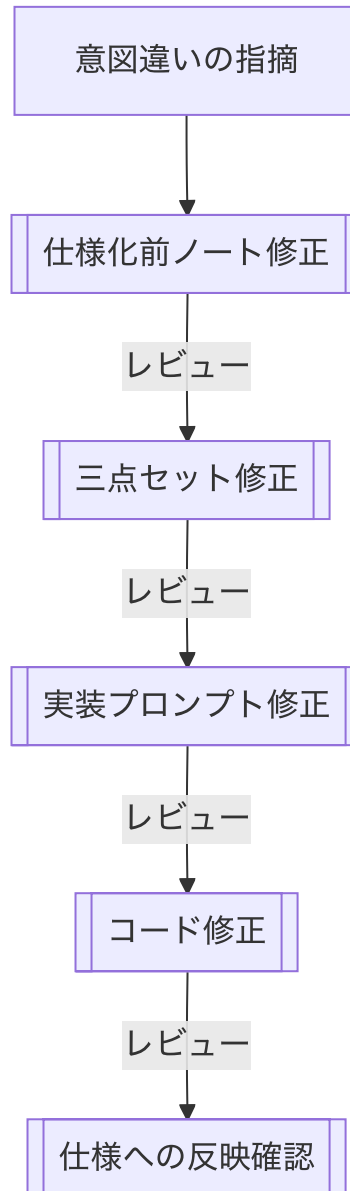
- UI とビジネスロジックを分離する。
- 入力検証や業務ルールはビジネスレイヤーに置く。
- UI は入力、操作、表示、エラー表示を担当する。
- 三点セットにない仕様を勝手に追加しない。
- 未決事項が実装に影響する場合は、実装前に確認する。
- `implementation_prompt.md` が使える状態でない場合は、実装へ進まない。

実装後は、実装結果が実装プロンプトと三点セットに従っているかを確認する。仕様外の機能追加、未決事項の仮決め、エラー条件の抜け、テスト不足、仕様への反映漏れがないかを見る。

6. 意図違いを修正する

実装結果が意図と違う場合、いきなりコードを修正しない。また、三点セットだけを先に修正しない。

修正は次の順番で行う。



禁止する進め方:

- 意図違いを指摘された直後にコードだけを修正する。
- `source_memo.md` を更新せず、三点セットだけを修正する。
- 実装プロンプトを更新せず、コードだけを修正する。
- 三点セットにない仕様をコードへ直接追加する。

- 仮決めを仕様へ戻さず、コードだけに残す。

AIへの指示例

最初に読ませる:

app-generation-workflow/three_key_documents_workflow.md を読んでください。

仕様化前ノートを作る:

任意形式の要求 .txt をもとに、対象アプリ用ディレクトリを作成し、
templates/source_memo.md をコピーして仕様化前ノートを作成してください。
アプリ種別を1つ選び、代表例と期待結果も記載してください。

逆生成レビュー用の仕様化前ノートを作る:

既存実装、生成済み三点セット、またはコードをもとに、
templates/source_memo_reverse_review.md をコピーして source_memo.md を作成し
てください。
現行実装から読み取れる挙動、仕様として採用する挙動、バグ候補・要確認事項を分
けて記載してください。

三点セットを作る:

source_memo.md をもとに、UC仕様書、UI仕様書、ビジネスレイヤー仕様書を作成し
てください。
source_memo.md のアプリ種別を三点セットへ引き継ぎ、不要な任意パターンは対象
外として整理してください。
作成した三点セットは草案として扱い、実装プロンプト作成前にレビューしてくださ
い。

実装プロンプトを作る:

レビュー済み三点セットを上位仕様として、指定した実装方式の実装プロンプトを作
成してください。
実装プロンプトが使える状態になっていない場合は、実装へ進まず警告してくださ
い。

意図違いを修正する:

実装結果が意図と違います。コードを直接修正せず、まず `source_memo.md` のどこを修正すべきか確認してください。